PROJECT AND TEAM INFORMATION

Project Title

Secure Process Scheduler with Malicious Job Detection and Persistent Database Queue Scheduling Job database

Student / Team Information

Team Name: *Team #*	*JobSentinel* *OSDBMS-V-2026-T172*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Kala Shivansh* *2026123* *24021777@geu.ac.in* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Khan Aabish* *2025001* *240211520@geu.ac.in* 

Team member 3 *(Last Name, name: student ID: email, picture):*	*Ghildiyal Anchita* *2025073* *240221675@geu.ac.in* 
Team member 4 *(Last Name, name: student ID: email, picture):*	*Negi Priyanshu* *2026092* 24011078@geu.ac.in 

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

(Describe the problem you want to solve and why it is important. Max 300 words).

Modern operating systems run several processes and jobs at once and need to schedule which process should run, when, and how to allocate the system resources. Old process schedulers concentrate mostly on such attributes of jobs as priority, waiting time, requirements to CPU usage, and time of job arrival while security issues are solved separately.

On the other hand, jobs residing only in memory can be lost after failure of scheduler or worker machine which causes both reliability and security problems for systems that work with valuable or even malicious jobs.

Thus, the proposed project is Secure Process Scheduler with Malicious Job Detection and Persistent Database Queue. It will combine ideas of OS process scheduling, database management and security. Jobs will be stored in PostgreSQL database, checked for malice, scheduled depending on their priority and availability of required system resources and executed in secure environment.

The project will show applications of:

- process scheduling,
- database management,
- concurrency control,
- persistent queue,
- resource management,
- secure process execution,
- runtime monitoring.

State of the Art / Current solution (1 pt)

(Describe how the problem is solved today (if it is). Max 200 words).

Traditional operating systems use scheduling algorithms such as:

- FCFS – executes processes according to arrival order.
- SJF – prioritizes processes with shorter CPU bursts.
- Round Robin – provides time-sharing using a time quantum.
- Priority Scheduling – executes higher-priority processes first.

Modern job-processing systems also provide persistent queues, worker management, retries, and database-backed task states.

However, a conventional educational process scheduler generally does not combine persistent database-based queue management with security-aware scheduling and malicious-job detection.

Existing scheduling approaches primarily answer:

Which job should execute next?

The proposed system extends this concept to:

Which job should execute next, and is it safe to execute?

The project therefore combines OS scheduling, DBMS persistence, and security controls in one system.

Project Goals and Milestones (2 pts)

(Describe the project general goals. Include initial milestones as well any other milestones. Max 300 words).

Here's what this project's all about: building a process scheduler that's both secure and dependable, using classic OS scheduling techniques with jobs stored safely in PostgreSQL.

Here's the plan:
- Include FCFS, SJF, Round Robin, and Priority scheduling algorithms.
- Store all job information in the PostgreSQL database.
- Catch suspicious jobs with rule-based checks.
- Score risks, and quarantine anything sketchy.
- Handle lots of workers running jobs at the same time, without missing a beat.
- Make sure transactions are solid, add locking to avoid conflicts, and recover gracefully from crashes or failed jobs.
- Log everything—both what gets executed and any security events along the way.

As for the milestones:

Phase 1: Set up the database and job management system.
Phase 2: Build out the scheduling algorithms.
Phase 3: Create the persistent queue and get workers processing.
Phase 4: Add the security and detection features.
Phase 5: Build a sandbox and set up monitoring.
Phase 6: Focus on recovery and retry mechanisms.
Phase 7: Run all the tests and build the dashboard.
Phase 8: Pull it all together in documentation and prep the final presentation.

Project Approach (3 pts)

(Describe how you plan to articulate and design a solution. Including platforms and technologies that you will use. Max 300 words).

Project Approach

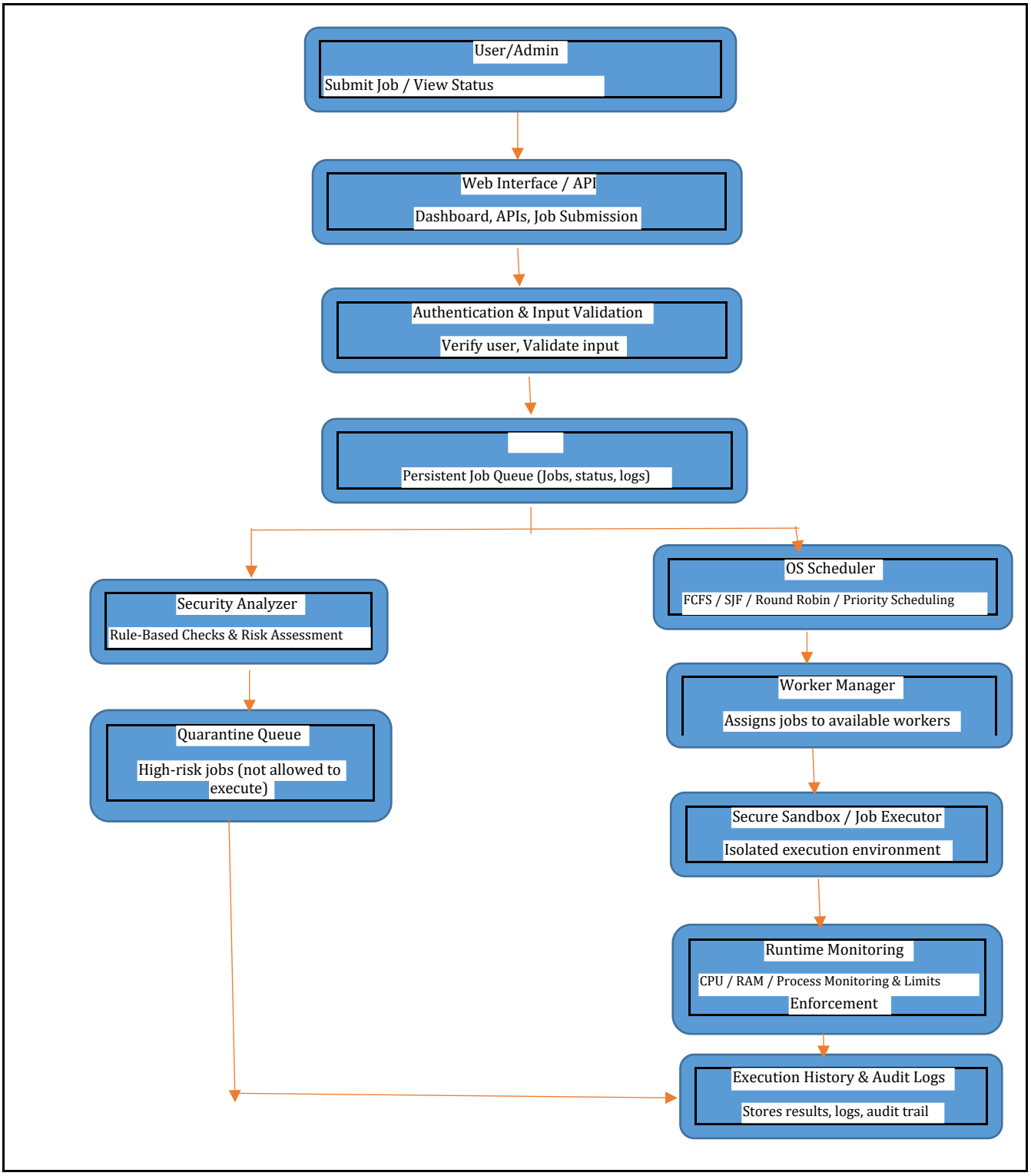
Technologies :

- I'll use Python or C++ for the main language.
- FastAPI will power the backend.
- MySQL handles the database.
- psutil lets us monitor system resources.
- Jobs run safely inside Docker containers.

Design Approach

1. Every submitted job goes into a persistent MySQL queue, so nothing slips through the cracks.
2. Each job comes with its own details: priority, required resources, and current status.
3. I'll set up different schedulers—FCFS, SJF, Round Robin, and Priority Scheduling—to compare how they each work in practice.
4. Before a job runs, it has to pass a security check. Anything suspicious gets flagged with the right level of risk.
5. If a job's too risky, it doesn't run. Instead, it's put in quarantine for review.
6. Multiple workers process jobs at the same time, but I'll use transactions and locks in the database to keep things safe and consistent.
7. I'll monitor running jobs—checking CPU, memory, and how long they're going—so nothing gets out of hand.
8. If something breaks, there's a recovery plan: retries, crash recovery, and audit logs keep job management solid even when things go sideways.

SYSTEM ARCHITECTURE



Project Outcome / Deliverables (1 pts)

(Describe what are the outcomes / deliverables of the project. Max 200 words).

These are some of the features to expect in this project:

- An uninterrupted process scheduler that directly communicates with PostgreSQL.
- Complete support of classic algorithms such as FCFS, SJF, Round Robin and Priority Scheduling.
- The system examines all the processes, finds any abnormalities based on the rules and places suspicious ones in isolation.
- Processes execute securely, and the system monitors resource usage from the very beginning.
- You get several workers handling processes simultaneously, while measures will be taken to prevent any collisions.
- In case of errors, the system will make a second attempt, and it is provided with crash recovery.
- All the runs will be traceable with full history and audit logs.
- Real-time web dashboard is available for the users to observe the process.
- And finally, performance analysis will be provided.

Assumptions

(Describe the assumptions (if any) you are making to solve the problem. Max 100 words)

1. The system will initially function in a Linux based setup.
2. Job submissions will happen from the application and not through arbitrary shell access.
3. There will be support for the PostgreSQL database.
4. Jobs will execute in sandboxed/controlled environments.
5. The scheduler will function with a finite number of workers.
6. Limits will be set for CPU and memory usage per job.
7. Security analysis will rely mainly on known rules and observed behavior.
8. For testing suspicious jobs, harmless simulations will be used.

References

(Provide a list of resources or references you utilised for the completion of this deliverable. You may provide links).

1 Scheduling real-time security aware taks in fog networks Authors: Anil Singh Nitin Auluck Omer F. Rana Andrew C. Jones https://www.researchgate.net/publication/332860207_Scheduling_Real_Time_Security_Aware_tasks_in_Fog_Networks

2 Security Enhancement Sceduling Model for IoT- Based Smart Cities Machine Learning Method Authors: Anoj Kumar Dwivedi Sanjeev Kumar Prasad https://onlinelibrary.wiley.com/doi/epdf/10.1002/adc2.235